



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Diego Licardié **Carné:** 1251825

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es un puente que permite a dos sistemas comunicarse por internet usando reglas estándar de la web. Funciona como un mesero que toma tu pedido, se lo lleva al servidor y te regresa la respuesta

2. ¿Qué es un endpoint?

Es la ruta específica dentro de un servidor a la que acudes para pedir o enviar un dato concreto. Por ejemplo, la dirección exacta para consultar la lista de productos.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección web completa y general de un sitio. El endpoint es la ruta técnica final dentro de esa dirección que procesa un recurso específico.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

Son las acciones básicas: GET lee datos, POST crea datos nuevos, PUT/PATCH modifica datos existentes y DELETE los elimina.

5. ¿Qué es JSON y por qué es común en las APIs?

Es un formato de texto simple para organizar datos en parejas de clave y valor. Es popular porque es muy liviano, fácil de entender para las personas y rápido de procesar para las computadoras.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es la respuesta numérica del servidor sobre el resultado de tu petición. **200** significa éxito, **404** que no se encontró la página y **500** que el servidor falló.

7. ¿Qué información se puede encontrar en la respuesta de una API?

El código de estado (si salió bien o mal), las cabeceras (datos sobre la conexión) y el cuerpo del mensaje (la información solicitada en formato JSON).



8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Permite enviar peticiones y ver respuestas reales sin escribir código de cliente. Ahorra tiempo al detectar fallos y validar la lógica antes de construir la aplicación.

9. ¿Qué es y para qué sirve Swagger UI?

Es un manual visual e interactivo de una API en el navegador. Sirve para ver qué rutas existen y probarlas haciendo un par de clics sin complicación.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Funciona como una libreta de pruebas para enviar, analizar y guardar solicitudes HTTP. Ayuda al desarrollador a comprobar que cada parte de la API responda exactamente como debe.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET



Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:



- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.